

```

1  /* =====
2      spine-horizontal.js – Graph-Ansicht: horizontale Spine + Play-Steuerung
3
4      Alternative zur Kartenansicht: jedes Kapitel zeigt seine Orte als
5      waagrechte Zeitleiste, deren Kreisgrafiken per Play-Animation mitwachsen.
6      Umschalten über "Plan"/"Graph" im Kapitelregister, abspielen über den
7      Play-Button, synchron zum Ton aus sonifikation.js.
8
9      ACHTUNG Abhängigkeitszyklus mit sonifikation.js: beide greifen gegenseitig
10     zu. Trägt nur, weil alle Zugriffe zur Laufzeit passieren – kein
11     Top-Level-Initialisierer hier darf je eine fremde Funktion aufrufen.
12     ===== */
13
14     // --- Modulkapselung -----
15     // 14 von 25 Namen intern, 11 exportiert. Konvention: docs/architektur.md.
16     (function () {
17
18         // Zustand der Play-Animation.
19         let grafikSpielt = false; // läuft die Wachstums-Animation gerade?
20         let grafikStartZeit = 0; // millis() bei Play-Start (bzw. rechnerisch zurückversetzt bei
21         Resume)
22         let grafikFortschritt = 0; // 0..1, letzter berechneter Animationsstand (bleibt bei Pause
23         stehen)
24         // millis() beim Play-Klick: der Einstiegstext weicht hier dem Play statt dem
25         // Scrollen. null = noch kein Play in dieser Ansicht.
26         let grafikPlayAusblendStart = null;
27
28         // -----
29         // Spine in p5
30         // -----
31
32         // Spine-Daten: einmal berechnen und halten.
33         let spineEintraegep5 = []; // { typ, text, rv, stationIdx, kreisId }
34         let spineEintraegeKapitel = {}; // Cache je Kapitelnummer (02–18), lazy befüllt beim ersten Zoom
35
36         // Baut die beiden Caches auf. kapitelNr ist letzterZoomKapitel, nicht
37         // zoomedKapitel: das Panel braucht beim Ausblenden noch seine Daten.
38         function stelleSpineDatenBereit(kapitelNr) {
39             if (spineEintraegep5.length === 0 && stationenData.ortRuns) {
40                 spineEintraegep5 = baueSpineDaten(stationenData, ortRunsFuerSpine(stationenData));
41             }
42             if (kapitelNr && !spineEintraegeKapitel[kapitelNr]) {
43                 let daten = datenFuerKapitel(kapitelNr);
44                 if (daten && daten.ortRuns) {
45                     spineEintraegeKapitel[kapitelNr] = baueSpineDaten(daten, ortRunsFuerSpine(daten));
46                 }
47             }
48         }
49
50         // Ohne Kapitelnummer die Einträge von Kapitel 1, sonst die des gezoomten
51         // Kapitels. Hält die beiden Caches modulintern.
52         function spineEintraegeFuer(kapitelNr) {
53             return kapitelNr ? spineEintraegeKapitel[kapitelNr] : spineEintraegep5;
54         }
55
56         // Menübalken "Plan"/"Graph". Jeder Wechsel in die Graph-Ansicht beginnt bei
57         // 0, die Animation muss per Play gestartet werden.
58         function setzeKapitelAnsichtModus(modus) {
59             if (kapitelAnsichtsModus === modus) return; // nur gelesen, nicht geschrieben
60             setzeAnsichtsModus(modus); // sketch.js
61             setzeGrafikZurueck(); // eigener Zustand, siehe unten
62         }
63
64         // Setzt den Play-Zustand zurück und stoppt den Ton mit – sonst lief er
65         // verwaist weiter. Gerufen bei Modus- und bei Kapitelwechsel.
66         function setzeGrafikZurueck() {

```

```

65     if (sonifikationSpieltGerade) beendeSonifikationAudio();
66     grafikSpielt = false;
67     grafikFortschritt = 0;
68     grafikPlayAusblendStart = null;
69 }
70
71 // Kapitel 1 nutzt die Dauer des Audiostücks, 02–18 dieselbe Geschwindigkeit
72 // pro Spine-Eintrag – sonst wirken kurze Kapitel hastig durchgespult.
73 function aktuelleGrafikAnimationDauer() {
74     // Im Elementmodell zählt die Zahl der Klänge, nicht die der Orte: Kapitel 2
75     // spielt an einem einzigen Ort und bekäme über die Ortsformel unten 2,6 s
76     // für 118 Elemente. Die Sonifikation gibt null zurück, wenn sie nicht
77     // zuständig ist – dann gilt weiter die Ortsformel.
78     let ausElementen = typeof sonifikationElementDauerMs === 'function'
79         ? sonifikationElementDauerMs(zoomedKapitel) : null;
80     if (ausElementen) return ausElementen;
81
82     if (!zoomedKapitel) return SONIFIKATION_GESAMTDAUER_SEK * 1000;
83     let n1 = spineEintraegep5.length;
84     let dauerProSchritt = (SONIFIKATION_GESAMTDAUER_SEK * 1000) / (n1 - 1 || 1);
85     let eintraege = spineEintraegeKapitel[zoomedKapitel];
86     let ni = eintraege ? eintraege.length : 1;
87     return dauerProSchritt * (ni - 1 || 1);
88 }
89
90 // Play/Pause der Spine-Animation, mit Ton synchron.
91 // ACHTUNG die Spine setzt bei Pause->Play fort, der Ton beginnt neu: Strudel
92 // kann nicht an einer beliebigen Stelle einsteigen.
93 function toggleGrafikPlay() {
94     if (grafikFortschritt >= 1) grafikFortschritt = 0; // am Ende: von vorn
95     grafikSpielt = !grafikSpielt;
96     if (grafikSpielt) {
97         // Nur beim ERSTEN Start merken, sonst blendet Pause/Weiter erneut aus.
98         if (grafikPlayAusblendStart === null) grafikPlayAusblendStart = millis();
99         grafikStartZeit = millis() - grafikFortschritt * aktuelleGrafikAnimationDauer();
100         spieleSonifikationFuer(zoomedKapitel);
101     } else if (sonifikationSpieltGerade) {
102         beendeSonifikationAudio();
103     }
104 }
105
106 function aktualisiereGrafikFortschritt() {
107     if (!grafikSpielt) return;
108     grafikFortschritt = constrain((millis() - grafikStartZeit) / aktuelleGrafikAnimationDauer(), 0,
109 1);
110     if (grafikFortschritt >= 1) grafikSpielt = false; // Ende erreicht, Button springt zurück auf Play
111 }
112
113 // Fester Abstand je Ortspunkt, Gesamtbreite ist n * Abstand; reicht der
114 // Platz nicht, wird gestaucht. Rechts hält der Rand das Kapitelregister frei
115 // (5vw), links steht nur Luft – beide gleich breit, damit die Spine mittig
116 // im Bild bleibt.
117 const SPINE_PUNKT_ABSTAND = 70;
118 const SPINE_RAND_LINKS = 200;
119 const SPINE_RAND_RECHTS = 200;
120 // Vertikale Linie vom Ortspunkt nach unten zur (horizontalen) Beschriftung.
121 const SPINE_LABEL_LINIE_LAENGE = 16;
122 const SPINE_LABEL_TEXT_ABSTAND = 6;
123
124 // Label-Zeilen werden EINMAL je Kapitel und Breite aus dem Endstand
125 // berechnet, damit sie beim Scrollen stillstehen.
126 const SPINE_LABEL_HOEHE = 16;
127 const SPINE_LABEL_ZEILEN_ABSTAND = 30;
128 // Freizuhaltender Rand für die vertikale Lage der Spine. Unten mehr wegen
129 // Play-Button und Scroll-Fortschrittsbalken.
130 const SPINE_RAND_OBEN = 24;

```

```

130 const SPINE_RAND_UNTEN = 76;
131 const spineLayoutCache = new WeakMap(); // eintraege-Array -> { breite, hoehe, versatz, breiten,
    linienY }
132
133 function spineLayout(eintraege, daten, abstand, startX) {
134     let vorhanden = spineLayoutCache.get(eintraege);
135     if (vorhanden && vorhanden.breite === width && vorhanden.hoehe === height) return vorhanden;
136
137     // Grösster Kreisradius am Kapitelende. Alle Labels liegen darunter, damit
138     // kein später gewachsener Kreis sie überdeckt.
139     let letzterIndex = daten.annotationen.length - 1;
140     let maxRadius = 0;
141     eintraege.forEach(e => {
142         if (e.typ === 'rueckkehr') return; // zeichnet keinen eigenen Kreis
143         let bc = zaehleAnnotationenLiveNachOrtBasis(wohnungFilterFuerOrt(e.ortBasis), letzterIndex,
    daten);
144         maxRadius = Math.max(maxRadius, groessterKreisRadius(bc));
145     });
146
147     // Höchster Rückkehr-Bogen: Halbkreis über der Linie, Radius ist der halbe
148     // Abstand zum Ursprungskreis. Bestimmt, wie tief die Linie liegen muss.
149     let maxBogen = 0;
150     eintraege.forEach((e, i) => {
151         if (e.typ !== 'rueckkehr') return;
152         maxBogen = Math.max(maxBogen, Math.abs(i - e.zielIndex) * abstand / 2);
153     });
154
155     // Dieselbe Schrift wie beim Zeichnen, sonst stimmen die Breiten nicht.
156     textFont(SHRIFT_SANS);
157     textStyle(BOLD);
158     textSize(13);
159
160     // Jedes Label in die oberste Zeile, in der rechts noch Platz ist.
161     let zeilenEnde = [];
162     let versatz = [];
163     let breiten = [];
164     eintraege.forEach((e, i) => {
165         let x = startX + i * abstand;
166         breiten[i] = textWidth(e.text);
167         let halbeBreite = breiten[i] / 2 + 4;
168         let zeile = zeilenEnde.findIndex(ende => x - halbeBreite > ende);
169         if (zeile === -1) {
170             zeile = zeilenEnde.length;
171             zeilenEnde.push(-Infinity);
172         }
173         zeilenEnde[zeile] = x + halbeBreite;
174         versatz[i] = maxRadius + SPINE_LABEL_LINIE_LAENGE + zeile * (SPINE_LABEL_HOEHE +
    SPINE_LABEL_ZEILEN_ABSTAND);
175     });
176
177     // Linie so weit nach unten, dass oben Bögen und Kreise Platz haben und
178     // unten die Labels. Passt beides, bleibt sie mittig.
179     let oben = Math.max(maxRadius, maxBogen);
180     let unten = Math.max(...versatz) + SPINE_LABEL_TEXT_ABSTAND + SPINE_LABEL_HOEHE;
181     let frei = height - SPINE_RAND_OBEN - SPINE_RAND_UNTEN;
182     let linienY;
183     if (oben + unten <= frei) {
184         linienY = constrain(height / 2, SPINE_RAND_OBEN + oben, height - SPINE_RAND_UNTEN - unten);
185     } else {
186         // Sehr niedriges Fenster: Überstand proportional auf beide Seiten.
187         linienY = SPINE_RAND_OBEN + frei * (oben / (oben + unten));
188     }
189
190     let ergebnis = { breite: width, hoehe: height, versatz, breiten, linienY };
191     spineLayoutCache.set(eintraege, ergebnis);
192     return ergebnis;

```

```

193 }
194
195 // Zentrierte Zeitleiste, per fortschritt (0..1) enthüllt. Alle Kreise teilen
196 // einen Spielkopf; eine Rückkehr bekommt nur einen Bogen, keinen zweiten Kreis.
197 function zeichneSpineHorizontal(eintraege, fortschritt, daten = stationenData) {
198   if (!eintraege.length) return;
199
200   let n = eintraege.length;
201   let verfuegbareBreite = width - SPINE_RAND_LINKS - SPINE_RAND_RECHTS;
202   let abstand = n > 1 ? Math.min(SPINE_PUNKT_ABSTAND, verfuegbareBreite / (n - 1)) :
SPINE_PUNKT_ABSTAND;
203   let startX = SPINE_RAND_LINKS + (verfuegbareBreite - (n - 1) * abstand) / 2;
204   // Lage und Label-Anordnung kommen fertig aus spineLayout (gecacht).
205   let layout = spineLayout(eintraege, daten, abstand, startX);
206   let linieY = layout.linienY;
207
208   // Playhead entlang der n Einträge. Eintrag 0 ist schon vor dem Play da.
209   let position = fortschritt * (n - 1 || 1);
210
211   // Interpoliert zwischen den revealIndex-Werten. Letzter Wegpunkt ist das
212   // Ende aller Annotationen, sonst wird der letzte Kreis nie voll.
213   let rvWegpunkte = eintraege.map(e => e.rv);
214   rvWegpunkte[n - 1] = daten.annotationen.length - 1;
215   let globalAnnIndex;
216   if (n === 1) {
217     // Einziger Eintrag (z.B. Kapitel 2): ab -1 interpolieren, sonst steht der
218     // Kreis ab fortschritt=0 sofort voll.
219     globalAnnIndex = Math.round(lerp(-1, rvWegpunkte[0], fortschritt));
220   } else {
221     let i0 = Math.min(n - 1, Math.floor(position));
222     let i1 = Math.min(n - 1, i0 + 1);
223     globalAnnIndex = Math.round(lerp(rvWegpunkte[i0], rvWegpunkte[i1], position - i0));
224   }
225
226   if (position > 0) {
227     noFill();
228     stroke(ROUTE_COLOR_RGB.r, ROUTE_COLOR_RGB.g, ROUTE_COLOR_RGB.b, 255);
229     strokeWeight(2);
230     line(startX, linieY, startX + Math.min(n - 1, position) * abstand, linieY);
231   }
232
233   // Rückkehr-Bögen zuerst, damit sie unter Kreisen und Punkten liegen.
234   eintraege.forEach((e, i) => {
235     if (e.typ !== 'rueckkehr') return;
236     let alphaSkala = constrain(position - (i - 1), 0, 1);
237     if (alphaSkala <= 0) return;
238     let x = startX + i * abstand;
239     let zielX = startX + e.zielIndex * abstand;
240     // Canvas-Pfad statt p5s arc(), siehe ACHTUNG unten bei fillText.
241     drawingContext.strokeStyle = `rgba(${ROUTE_COLOR_RGB.r}, ${ROUTE_COLOR_RGB.g},
${ROUTE_COLOR_RGB.b}, ${alphaSkala})`;
242     drawingContext.lineWidth = 2;
243     drawingContext.beginPath();
244     drawingContext.arc((x + zielX) / 2, linieY, Math.abs(x - zielX) / 2, PI, TWO_PI);
245     drawingContext.stroke();
246   });
247
248   // ACHTUNG nach Grösse zeichnen, nicht in Zeitleisten-Reihenfolge: bei
249   // engem SPINE_PUNKT_ABSTAND deckt ein später gezeichneter kleinerer Kreis
250   // sonst einen grösseren Nachbarn an.
251   let kreisDaten = [];
252   eintraege.forEach((e, i) => {
253     if (e.typ === 'rueckkehr') return;
254     let alphaSkala = constrain(position - (i - 1), 0, 1);
255     if (alphaSkala <= 0) return;
256     let x = startX + i * abstand;

```

```

257 // Ein Scan für beides: Kreisflächen und F-Wert-Punkte.
258 let filter = wohnungFilterFuerOrt(e.ortBasis);
259 let treffer = sammleAnnotationenNachOrtBasis(filter, globalAnnIndex, daten);
260 let bc = zaehleBandCounts(treffer);
261 let fwertAnnotationen = treffer.filter(a => a.hasFwert);
262 kreisDaten.push({ i, x, bc, fwertAnnotationen, radius: 0 });
263 });
264
265 // Grösse vorab bestimmen und sortieren. groessterKreisRadius ist dieselbe
266 // Funktion, die zeichneKreiseFuerRun intern nutzt.
267 kreisDaten.forEach(k => { k.radius = groessterKreisRadius(k.bc); });
268 kreisDaten.sort((a, b) => b.radius - a.radius);
269
270 let radiusNachIndex = new Map();
271 kreisDaten.forEach(k => {
272   // winkel PI: Halbkreise oben/unten statt links/rechts, positiv oben.
273   // F-Wert-Punkte bekommen denselben Winkel.
274   zeichneKreiseFuerRun(k.x, linieY, k.bc, 1, PI);
275   merkeKreis(k.x, linieY, k.bc, k.radius,
276     zeichneFwertPunkte(k.x, linieY, k.radius, k.fwertAnnotationen, 1));
277   radiusNachIndex.set(k.i, k.radius);
278 });
279
280 textFont(SCHRIFT_SANS);
281 textStyle(BOLD);
282 textSize(13);
283 textAlign(CENTER, TOP);
284
285 // Durchgang 1: Ortspunkte und Zuführungslinien – nach allen Kreisen, damit
286 // sie nie unter einem Nachbarkreis verschwinden.
287 eintraege.forEach((e, i) => {
288   let alphaSkala = constrain(position - (i - 1), 0, 1);
289   if (alphaSkala <= 0) return;
290   let x = startX + i * abstand;
291   let radius = radiusNachIndex.get(i) || 0;
292
293   // Ortspunkt als Canvas-Pfad, siehe ACHTUNG unten bei fillText.
294   drawingContext.fillStyle = `rgba(0, 0, 0, ${alphaSkala})`;
295   drawingContext.beginPath();
296   drawingContext.arc(x, linieY, 2.5, 0, TWO_PI);
297   drawingContext.fill();
298
299   // Zuführungslinie vom Kreisrand hinunter zur festen Label-Zeile.
300   stroke(0, 110 * alphaSkala);
301   strokeWeight(1);
302   line(x, linieY + (radius > 0 ? radius : 4), x, linieY + layout.versatz[i]);
303   noStroke();
304 });
305
306 // Durchgang 2: Beschriftungen – nach allen Linien, damit keine Linie ein
307 // fremdes Label durchschneidet. Jedes Label stellt sich dafür frei.
308 eintraege.forEach((e, i) => {
309   let alphaSkala = constrain(position - (i - 1), 0, 1);
310   if (alphaSkala <= 0) return;
311   let x = startX + i * abstand;
312   let textY = linieY + layout.versatz[i] + SPINE_LABEL_TEXT_ABSTAND;
313   let breite = layout.breiten[i];
314
315   drawingContext.fillStyle = `rgba(226, 230, 225, ${alphaSkala})`;
316   drawingContext.fillRect(x - breite / 2 - 3, textY - 2, breite + 6, SPINE_LABEL_HOEHE - 2);
317
318   // ACHTUNG p5s text()/arc()/ellipse() bleiben bei laufender Animation
319   // manchmal unsichtbar, obwohl der Context korrekt gesetzt ist. Deshalb
320   // wird hier und an drei Stellen oben direkt über drawingContext
321   // gezeichnet. Ursache ungeklärt, nur umgangen.
322   drawingContext.fillStyle = `rgba(26, 26, 26, ${alphaSkala})`;

```

```
323     drawingContext.fillText(e.text, x, textY);
324   });
325
326   textStyle(NORMAL);
327 }
328
329
330 // --- Export -----
331 // Acht Funktionen als Wert.
332 window.setzeKapitelAnsichtModus = setzeKapitelAnsichtModus;
333 window.setzeGrafikZurueck = setzeGrafikZurueck;
334 window.toggleGrafikPlay = toggleGrafikPlay;
335 window.aktualisiereGrafikFortschritt = aktualisiereGrafikFortschritt;
336 window.aktuelleGrafikAnimationDauer = aktuelleGrafikAnimationDauer;
337 window.stelleSpineDatenBereit = stelleSpineDatenBereit;
338 window.spineEintraegeFuer = spineEintraegeFuer;
339 window.zeichneSpineHorizontal = zeichneSpineHorizontal;
340
341 // Lesebindung statt Wertkopie: sonst sieht draw() dauerhaft false/0/null.
342 ['grafikSpielt', 'grafikFortschritt', 'grafikPlayAusblendStart'].forEach(function (name) {
343   Object.defineProperty(window, name, {
344     get: function () {
345       return name === 'grafikSpielt' ? grafikSpielt
346         : name === 'grafikFortschritt' ? grafikFortschritt
347         : grafikPlayAusblendStart;
348     },
349     configurable: true,
350   });
351 });
352
353 // Die beiden Caches gehen nicht hinaus, gelesen wird über
354 // spineEintraegeFuer() – sonst bliebe die Objektreferenz beschreibbar.
355
356 })(); // Ende der Modulkapselung, siehe Kommentar oben
```